```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>

// ── Pin Definitions ──────────────────────────────────────────
#define MQ2_ANALOG_PIN   A0
#define FLAME_PIN        8
#define BUZZER_PIN       9

// ── Smoke Thresholds ─────────────────────────────────────────
#define SMOKE_LOW_MIN    300
#define SMOKE_MED_MIN    500
#define SMOKE_HIGH_MIN   700

// ── Alert Levels ─────────────────────────────────────────────
enum AlertLevel { SAFE, LOW_ALERT, MEDIUM_ALERT, HIGH_ALERT };

// ── GSM SIM900A (TX->D10, RX->D11) ──────────────────────────
SoftwareSerial gsm(10, 11);

// ── LCD I2C (try 0x3F if screen is blank) ────────────────────
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ── Your Phone Number ────────────────────────────────────────
const String PHONE_NUMBER = "+919080535934";

// ── State Variables ──────────────────────────────────────────
AlertLevel lastAlertSent = SAFE;
unsigned long lastCheck  = 0;

// ── Custom LCD Characters ────────────────────────────────────
byte bellChar[8] = {
  0b00100,
  0b01110,
  0b01110,
  0b01110,
  0b11111,
  0b00000,
  0b00100,
  0b00000
};

byte flameChar[8] = {
  0b00010,
```

```
  0b00110,
  0b01110,
  0b11110,
  0b11111,
  0b01110,
  0b00110,
  0b00010
};

//
————————————————————————————————————————————————————————————————————————————————
—
//  SETUP
//
————————————————————————————————————————————————————————————————————————————————
—
void setup() {
  Serial.begin(9600);
  gsm.begin(9600);

  pinMode(FLAME_PIN,  INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  noTone(BUZZER_PIN);

  // LCD init
  lcd.init();
  lcd.backlight();
  lcd.createChar(0, bellChar);
  lcd.createChar(1, flameChar);

  // Splash screen
  lcd.setCursor(0, 0);
  lcd.print("  FIRE & SMOKE  ");
  lcd.setCursor(0, 1);
  lcd.print(" ALERT SYSTEM   ");
  delay(2500);
  lcd.clear();

  // GSM init
  initGSM();

  lcd.setCursor(0, 0);
  lcd.print("System Ready!   ");
  lcd.setCursor(0, 1);
```

```
  lcd.print("Monitoring...   ");
  delay(1500);
}

//
──────────────────────────────────────────────────────────────────────────────
─
//  LOOP
//
──────────────────────────────────────────────────────────────────────────────
─
void loop() {
  if (millis() - lastCheck < 1000) return;
  lastCheck = millis();

  // Read sensors
  int  smokeADC      = analogRead(MQ2_ANALOG_PIN);
  bool flameDetected = (digitalRead(FLAME_PIN) == LOW);

  Serial.print("Smoke ADC: ");
  Serial.print(smokeADC);
  Serial.print(" | Flame: ");
  Serial.println(flameDetected ? "YES" : "NO");

  // Determine smoke level
  AlertLevel level;
  if      (smokeADC >= SMOKE_HIGH_MIN) level = HIGH_ALERT;
  else if (smokeADC >= SMOKE_MED_MIN)  level = MEDIUM_ALERT;
  else if (smokeADC >= SMOKE_LOW_MIN)  level = LOW_ALERT;
  else                                 level = SAFE;

  // If flame detected, immediately escalate to HIGH ALERT
  if (flameDetected) level = HIGH_ALERT;

  // Update LCD display
  updateLCD(smokeADC, flameDetected, level);

  // Handle buzzer
  handleBuzzer(level);

  // GSM Actions based on level
  if (level == SAFE) {
    lastAlertSent = SAFE;
  }
```

```
  else if (level == LOW_ALERT && lastAlertSent == SAFE) {
    sendSMS("LOW", smokeADC, flameDetected);
    lastAlertSent = LOW_ALERT;
  }
  else if (level == MEDIUM_ALERT && lastAlertSent < MEDIUM_ALERT) {
    sendSMS("MEDIUM", smokeADC, flameDetected);
    lastAlertSent = MEDIUM_ALERT;
  }
  else if (level == HIGH_ALERT && lastAlertSent < HIGH_ALERT) {
    sendSMS("HIGH", smokeADC, flameDetected);
    lastAlertSent = HIGH_ALERT;
    delay(30000);         // Wait 30s before allowing another SMS
    lastAlertSent = SAFE; // Reset so it can alert again if still dangerous
  }
}

//
_____________________________________________________________________________
_
//  LCD DISPLAY
//
_____________________________________________________________________________
_
void updateLCD(int smokeADC, bool flame, AlertLevel level) {
  lcd.clear();

  switch (level) {

    case SAFE:
      lcd.setCursor(0, 0);
      lcd.print("Status : SAFE   ");
      lcd.setCursor(0, 1);
      lcd.print("Smoke:");
      lcd.print(smokeADC);
      lcd.print(" Flm:");
      lcd.print(flame ? "Y" : "N");
      break;

    case LOW_ALERT:
      lcd.setCursor(0, 0);
      lcd.write(byte(0));
      lcd.print(" LOW  ALERT!    ");
      lcd.setCursor(0, 1);
      lcd.print("Smoke:LOW  ");
```

```
      lcd.print(smokeADC);
      lcd.print(flame ? " F:Y" : " F:N");
      break;

    case MEDIUM_ALERT:
      lcd.setCursor(0, 0);
      lcd.write(byte(0));
      lcd.print(" MEDIUM ALERT!  ");
      lcd.setCursor(0, 1);
      lcd.print("Smoke:MED  ");
      lcd.print(smokeADC);
      lcd.print(flame ? " F:Y" : " F:N");
      break;

    case HIGH_ALERT:
      lcd.setCursor(0, 0);
      lcd.write(byte(1));
      lcd.print(" HIGH ALERT!!! ");
      lcd.setCursor(0, 1);
      lcd.print("Smoke:HIGH ");
      lcd.print(smokeADC);
      lcd.print(flame ? " F:Y" : " F:N");
      break;
  }
}

//
_____________________________________________________________________________
_
//  BUZZER
//
_____________________________________________________________________________
_
void handleBuzzer(AlertLevel level) {
  switch (level) {

    case SAFE:
      noTone(BUZZER_PIN);
      break;

    case LOW_ALERT:
      tone(BUZZER_PIN, 800, 200);
      break;
```

```
    case MEDIUM_ALERT:
      tone(BUZZER_PIN, 1000, 300);
      delay(400);
      tone(BUZZER_PIN, 1000, 300);
      break;

    case HIGH_ALERT:
      tone(BUZZER_PIN, 1500, 500);
      delay(600);
      tone(BUZZER_PIN, 1500, 500);
      break;
  }
}

//
————————————————————————————————————————————————————————————————————————————————
—
//  GSM INIT
//
————————————————————————————————————————————————————————————————————————————————
—
void initGSM() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("GSM Initializing");
  Serial.println("Initializing GSM...");

  delay(3000);

  gsm.println("AT");
  delay(1000);

  gsm.println("AT+CMGF=1");        // SMS text mode
  delay(1000);

  gsm.println("AT+CNMI=1,2,0,0,0");
  delay(1000);

  // Check network registration
  gsm.println("AT+CREG?");
  delay(2000);

  lcd.setCursor(0, 1);
  lcd.print("GSM Ready!      ");
```

```
  Serial.println("GSM Ready.");
  delay(1000);
}

//
──────────────────────────────────────────────────────────────────────────────
──
//  SEND SMS  (ALL levels — LOW, MEDIUM, HIGH)
//
──────────────────────────────────────────────────────────────────────────────
──
void sendSMS(String lvl, int smokeADC, bool flame) {
  Serial.print("Sending SMS - Level: ");
  Serial.println(lvl);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Sending SMS...  ");
  lcd.setCursor(0, 1);
  lcd.print("Level: ");
  lcd.print(lvl);

  // Build message
  String msg = "[" + lvl + " ALERT] Fire & Smoke System\n";
  msg += "Smoke Level : " + lvl + " (" + String(smokeADC) + ")\n";
  msg += "Flame       : ";
  msg += flame ? "DETECTED\n" : "Not Detected\n";
  if (lvl == "HIGH") {
    msg += "DANGER! Immediate action required!";
  } else {
    msg += "Please check immediately!";
  }

  gsm.println("AT+CMGF=1");
  delay(500);

  gsm.print("AT+CMGS=\"");
  gsm.print(PHONE_NUMBER);
  gsm.println("\"");
  delay(500);

  gsm.print(msg);
  delay(500);
```

```
  gsm.write(26);    // Ctrl+Z to send SMS
  delay(4000);

  Serial.println("SMS Sent!");

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SMS Sent!       ");
  lcd.setCursor(0, 1);
  lcd.print("Level: ");
  lcd.print(lvl);
  delay(2000);
}
```